

Stage 1 Execution Plan — Dataset Exploration, Analysis & Preparation

Time budget: 4 hours 25 minutes | Weightage: 15% | Team: all 4 members in parallel

Before anything: what “done” looks like (keep this visible)

By the end of this block, you need these 7 deliverables, exactly as the problem statement lists them: 1. Pedestrian instance distribution + dataset statistics (train/val/test counts, avg pedestrians/image) 2. Pedestrian scale distribution (Small $<32^2$, Medium 32^2-96^2 , Large $\geq 96^2$ pixels) 3. Sample RGB-Thermal image pairs 4. Visualization of ≥ 20 annotated RGB-Thermal pairs, side by side 5. RGB-Thermal alignment verification 6. Description/justification of any preprocessing applied (optional — thermal/contrast enhancement) 7. Written discussion of key dataset challenges (occlusion, low light, clutter, small pedestrians)

Everything below is built to produce exactly these 7 things, nothing extra, nothing skipped.

Folder Structure (set this up first, 5 minutes)

```
medhadrishiti-2026/
├── data/
│   ├── vtuvav_subset/          # downloaded dataset goes here, untouched/raw
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
├── stage1_analysis/
│   ├── scripts/
│   │   ├── 01_inspect_structure.py
│   │   ├── 02_instance_statistics.py
│   │   ├── 03_scale_distribution.py
│   │   ├── 04_visualize_pairs.py
│   │   └── 05_alignment_check.py
│   ├── outputs/
│   │   ├── visualizations/      # saved images go here
│   │   ├── statistics.json
│   │   └── challenges_notes.md
│   ├── requirements.txt
├── docs/
│   └── stage1_report_section.md  # this becomes part of your final 3-5 page report
```

Create this now (whoever’s laptop is fastest to set up), push it, everyone pulls before starting their piece.

STEP 0 (0:00-0:15) — Download + environment setup, whole team in parallel

One person (fastest internet): Download the VTUAV_subset from the provided Google Drive link immediately — this is the long pole, start it before anything else. While it downloads:

Everyone else, install these now:

```
pip install opencv-python matplotlib pandas numpy pycocotools pillow
```

`pycocotools` matters — given the problem statement mentions COCO-format predictions and COCO evaluation metrics, the annotations are very likely in COCO JSON format. We’ll confirm this in Step 1.

STEP 1 (0:15-0:35) — Inspect what you actually got (don’t assume, verify)

Owner: Shankar (he’s most familiar with dataset structures from earlier prep)

Write `01_inspect_structure.py` :

```
import os

data_root = "data/vtuvav_subset"

for split in ["train", "val", "test"]:
    split_path = os.path.join(data_root, split)
    if os.path.exists(split_path):
        print(f"\n=== {split} ===")
```

```
for item in os.listdir(split_path):
    print(" ", item)
```

Run it. This tells you the real folder layout — RGB images, thermal images, and annotations might be organized differently than assumed. **Look specifically for:** - Where RGB images live vs thermal images (often folders like `rgb/` and `thermal/` or `visible/` and `infrared/`) - Whether annotations are one JSON file per split (COCO-style) or one text file per image (YOLO-style)

If annotations are **COCO JSON**, open it and check the structure:

```
import json
with open("data/vtuav_subset/train/annotations.json") as f:
    coco_data = json.load(f)
print(coco_data.keys())          # should show: images, annotations, categories
print(coco_data["categories"])   # confirm "pedestrian" class exists
print(coco_data["annotations"][0]) # look at one example annotation
```

Message the group immediately with what you find — the exact folder names and annotation format — since everyone else's scripts depend on this.

STEP 2 (0:35-1:35) — Pedestrian instance distribution & statistics

Owner: Shashank

Once Step 1 confirms the format, write `02_instance_statistics.py`. If COCO format (most likely):

```
import json
import pandas as pd

def analyze_split(json_path, split_name):
    with open(json_path) as f:
        data = json.load(f)

    num_images = len(data["images"])
    num_instances = len(data["annotations"])
    avg_per_image = round(num_instances / num_images, 2) if num_images else 0

    print(f"\n=== {split_name} ===")
    print(f"Images: {num_images}")
    print(f"Pedestrian instances: {num_instances}")
    print(f"Avg pedestrians per image: {avg_per_image}")

    return {
        "split": split_name,
        "images": num_images,
        "instances": num_instances,
        "avg_per_image": avg_per_image
    }

results = []
results.append(analyze_split("data/vtuav_subset/train/annotations.json", "train"))
results.append(analyze_split("data/vtuav_subset/val/annotations.json", "val"))
results.append(analyze_split("data/vtuav_subset/test/annotations.json", "test"))

df = pd.DataFrame(results)
df.to_csv("stage1_analysis/outputs/instance_statistics.csv", index=False)
print(df)
```

Note: adjust file paths/names once Step 1 confirms actual filenames — they may not literally be called `annotations.json`, could be `train.json`, `_annotations.coco.json`, etc.

Cross-check against the problem statement's official numbers (1200 train / 300 val / 200 test image pairs) — if your image counts don't match, something's off in how you're reading the data, fix before moving on.

STEP 3 (1:35-2:15) — Pedestrian scale distribution

Owner: Shashank (continues) or Sanjana if Shashank needs to move to visualization

Use the exact thresholds from the problem statement — this matters, don't approximate: - Small: $\text{area} < 32^2$ (1024 px²) - Medium: $32^2 \leq \text{area} < 96^2$ (1024 to 9216 px²) - Large: $\text{area} \geq 96^2$ (9216 px²)

```
import json
```

```
def scale_distribution(json_path, split_name):
    with open(json_path) as f:
        data = json.load(f)

    small, medium, large = 0, 0, 0
    for ann in data["annotations"]:
        w, h = ann["bbox"][2], ann["bbox"][3] # COCO bbox format: [x, y, width, height]
        area = w * h
        if area < 32**2:
            small += 1
        elif area < 96**2:
            medium += 1
        else:
            large += 1

    total = small + medium + large
    print(f"\n=== {split_name} scale distribution ===")
    print(f"Small: {small} ({round(100*small/total,1)}%)")
    print(f"Medium: {medium} ({round(100*medium/total,1)}%)")
    print(f"Large: {large} ({round(100*large/total,1)}%)")

    return {"split": split_name, "small": small, "medium": medium, "large": large}

for split in ["train", "val", "test"]:
    scale_distribution(f"data/vtuav_subset/{split}/annotations.json", split)
```

This will very likely show a heavy skew toward “small” — that’s expected and directly matches the problem statement’s stated challenge (“primary challenge is detection of small and tiny pedestrians”). **Note this finding explicitly for your report — it’s a direct setup for your Stage 3 fusion strategy justification later.**

STEP 4 (2:15-3:05) — Visualize ≥ 20 annotated RGB-Thermal pairs

Owner: Sanjana (she already knows image display/annotation drawing from frontend work — directly transferable skill)

```
import cv2
import json
import matplotlib.pyplot as plt
import os

def draw_boxes(image, annotations, image_id):
    img = image.copy()
    for ann in annotations:
        if ann["image_id"] == image_id:
            x, y, w, h = [int(v) for v in ann["bbox"]]
            cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    return img

with open("data/vtuav_subset/train/annotations.json") as f:
    coco = json.load(f)

os.makedirs("stage1_analysis/outputs/visualizations", exist_ok=True)

# Adjust these paths once Step 1 confirms actual RGB/thermal folder names
rgb_dir = "data/vtuav_subset/train/rgb"
thermal_dir = "data/vtuav_subset/train/thermal"

for i, img_info in enumerate(coco["images"][:20]): # first 20 images
    filename = img_info["file_name"]
    image_id = img_info["id"]

    rgb_img = cv2.imread(os.path.join(rgb_dir, filename))
    thermal_img = cv2.imread(os.path.join(thermal_dir, filename))

    rgb_annotated = draw_boxes(rgb_img, coco["annotations"], image_id)
    thermal_annotated = draw_boxes(thermal_img, coco["annotations"], image_id)

    fig, axes = plt.subplots(1, 2, figsize=(12, 6))
    axes[0].imshow(cv2.cvtColor(rgb_annotated, cv2.COLOR_BGR2RGB))
    axes[0].set_title("RGB")
    axes[0].axis("off")
    axes[1].imshow(cv2.cvtColor(thermal_annotated, cv2.COLOR_BGR2RGB))
    axes[1].set_title("Thermal")
```

```

axes[1].axis("off")

plt.savefig(f"stage1_analysis/outputs/visualizations/pair_{i:02d}.png", bbox_inches="tight")
plt.close()

print("Saved 20 annotated pairs to outputs/visualizations/")

```

This single script produces **two deliverables at once**: sample RGB-Thermal pairs (#3) and annotated visualization (#4).

STEP 5 (3:05-3:35) — RGB-Thermal alignment verification

Owner: Sanjana (continues) or You, if she needs a break

“Alignment verification” means: confirming the RGB and thermal images of the same pair are actually showing the same scene, same pedestrians, in the same positions (not shifted/misaligned).

Fastest method — visual overlay check:

```

import cv2
import numpy as np

def check_alignment(rgb_path, thermal_path, output_path):
    rgb = cv2.imread(rgb_path)
    thermal = cv2.imread(thermal_path)

    # Resize thermal to match RGB dimensions if they differ
    thermal_resized = cv2.resize(thermal, (rgb.shape[1], rgb.shape[0]))

    # Blend both images to visually check if edges/objects line up
    overlay = cv2.addWeighted(rgb, 0.5, thermal_resized, 0.5, 0)
    cv2.imwrite(output_path, overlay)

# Run on a handful of sample pairs
check_alignment(
    "data/vtuav_subset/train/rgb/sample1.jpg",
    "data/vtuav_subset/train/thermal/sample1.jpg",
    "stage1_analysis/outputs/visualizations/alignment_check_1.png"
)

```

Look at the output overlay — if pedestrian outlines from both modalities roughly line up, alignment is good. Do this for 3-5 sample pairs, note any visible misalignment (common in RGBT datasets due to slightly different camera positions) in your write-up.

STEP 6 (3:35-3:55) — Optional preprocessing (only if time allows — it’s marked optional in the problem statement)

Skip this if you’re behind schedule. If you have spare time: - Try a simple contrast enhancement on thermal images (`cv2.equalizeHist()`) and note whether it visually helps distinguish pedestrians - Document this as “explored but not applied to final pipeline” if you don’t have time to properly integrate it — partial credit for showing awareness is fine here, this is a low-weight optional item

STEP 7 (3:55-4:20) — Write up the challenges discussion + compile report section

Owner: You

While the team finishes their scripts, draft `docs/stage1_report_section.md` — this becomes a direct chunk of your final technical report:

```

## Stage 1: Dataset Exploration, Analysis, and Preparation

### Dataset Statistics
[Insert table from instance_statistics.csv]

### Pedestrian Scale Distribution
[Insert small/medium/large percentages per split]
Note: the dataset shows a strong skew toward small-scale pedestrians,
confirming the problem statement's emphasis on this as the primary detection challenge.

### RGB-Thermal Alignment
[1-2 sentences: alignment verified visually across sample pairs / note any misalignment observed]

### Key Challenges Identified

```

- Small/tiny pedestrian instances dominate the dataset (see scale distribution)
- [Add: occlusion cases observed in visualized pairs, if any]
- [Add: illumination variation observed, if any]
- [Add: any RGB-thermal misalignment observed]

Visualizations

See `stage1_analysis/outputs/visualizations/` for 20 annotated RGB-Thermal pairs.

Fill in the bracketed parts as results come in from the team.

STEP 8 (4:20-4:25) — Final compile and push

- Confirm all 7 deliverables exist in `stage1_analysis/outputs/`
- Push everything to the repo
- Quick team check-in: does the scale distribution and challenges write-up actually match what you saw in the visualizations? Fix any mismatches before moving to Stage 2

If something breaks (common issues, fast fixes)

Annotation format isn't COCO JSON: If Step 1 reveals a different format (e.g. VOC XML, or plain text), message the group immediately — the parsing logic in Steps 2-4 needs adjusting, but the overall plan and deliverables stay the same. Don't panic, just adapt the file-reading part.

Image filenames don't match between RGB/thermal/annotations: Very common in RGBT datasets — check if there's an ID-based mapping instead of matching filenames directly, look for a mapping file or consistent ID field in the JSON.

Running low on time: Deliverables #1, #2, and #4 (statistics, scale distribution, visualizations) are the highest-value, most concretely gradable items — prioritize finishing these fully even if #5/#6/#7 end up brief.